

Month 1 Implementation Plan

GOAL

Build a safe first version of the chatbot backend that we can test easily.

For each message, it follows the same clear path:

input -> risk check -> find evidence -> draft response -> safety check -> final output.

LAY EXPLANATION OF EACH COMPONENT

- FastAPI app (app/main.py): This is the front door of the chatbot. It receives user messages and sends back answers.
 - Schemas (app/schemas.py): These are the rules for data shape. They make sure requests and responses are in the right format.
 - Config (app/config.py): This stores basic app settings, like app name and environment.
 - Orchestrator state (app/orchestrator/state.py): This is the chatbot's memory for one conversation step. It tracks things like risk flags, evidence, and final response.
 - Orchestrator nodes (app/orchestrator/nodes.py): These are the step-by-step actions the chatbot takes (check risk, find evidence, write draft, apply safety rules).
 - Orchestrator graph (app/orchestrator/graph.py): This is the map that decides the order of steps in the workflow.
 - RAG service (app/services/rag.py): This finds helpful facts from trusted sources so the chatbot can give grounded answers.
 - Generator service (app/services/generator.py): This writes a draft response using the user question and the found evidence.
 - Policy service (app/services/policy.py): This is the safety checker. It can block unsafe replies and trigger escalation for urgent cases.
 - ****Tests (tests/*.py)****: These are automatic checks that prove the app routes and workflow are working correctly.
-

WEEK-BY-WEEK SCOPE

- Week 1: Create the project folders and core files. Add schemas and the /health route.
 - Week 2: Build the workflow state and LangGraph step nodes.
 - Week 3:
 - Add simple RAG and response generator service stubs.
 - Add safety policy logic and escalation behavior for risky messages.
 - Week 4: Add tests for API routes and workflow behavior.
-

EXACT FILES

- app/main.py – Runs the FastAPI app and API routes.

- `app/schemas.py` – Defines what request and response data should look like.
 - `app/config.py` – Stores basic app settings.
 - `app/orchestrator/state.py` – Holds workflow data for one chat step.
 - `app/orchestrator/nodes.py` – Contains each workflow step function.
 - `app/orchestrator/graph.py` – Connects step functions in the right order.
 - `app/services/rag.py` – Finds evidence (currently a starter stub).
 - `app/services/generator.py` – Creates a draft response (starter stub).
 - `app/services/policy.py` – Checks safety and decides if escalation is needed.
 - `tests/test_health.py` – Tests that health route works.
 - `tests/test_chat_route.py` – Tests chat endpoint behavior.
 - `tests/test_graph.py` – Tests workflow logic.
-

FASTAPI ENDPOINTS

- `GET /health`
 - Returns a simple status response to show the app is running.
 - `POST /api/v1/chat`
 - Input: `ChatRequest` (session ID + user message)
 - Output: `ChatResponse` (answer + citations + escalation info)
 - What it does:
 - creates the starting workflow state
 - runs the `LangGraph` steps in order
 - returns final response text, evidence citations, and escalation flag
-

LANGGRAPH NODES

- `ingest_input_node`:
 - Cleans the message and checks for basic risk keywords.
 - `retrieve_evidence_node`:
 - Calls the RAG service and adds evidence to state.
 - `generate_draft_node`:
 - Builds a draft response from the user message + evidence.
 - `policy_gate_node`:
 - Applies fixed safety rules and decides if escalation is needed.
 - `finalize_response_node`:
 - Packages the final response payload for the API.
-

MONTH 1 TESTS

- `test_health_endpoint_returns_ok` – Confirms the app reports healthy status.
- `test_chat_endpoint_returns_response` – Confirms normal chat returns a valid answer.
- `test_chat_endpoint_escalates_on_red_flag` – Confirms risky text triggers escalation.

- test_graph_generates_citations - Confirms workflow returns evidence citations.
 - test_policy_gate_blocks_unsafe_output - Confirms policy blocks unsafe behavior.
-

DEFINITION OF DONE

- API endpoints run locally without errors.
- Workflow follows the same predictable order each time.
- All Month 1 tests pass.
- Responses include both citations and escalation metadata.